

LU2IN006 - Rapport - Simulateur de CPU

rédigé par Kyrylo Hoviadin et Victor Ji - (Groupe 10)

Ce projet consiste à réaliser en langage C, un simulateur de CPU simplifié pouvant reproduire le fonctionnement d'un processeur exécutant des programmes en assembleur. Le système devra gérer des registres (AX, BX, ...), une mémoire segmentée (données, code, pile) et implémenter des modes d'adressage (immédiat, registre, direct et indirect). L'objectif principal repose sur un cycle d'instruction, dans lequel on récupère une instruction en mémoire, la décode, puis l'exécute en manipulant les registres et la mémoire. À noter qu'on a la possibilité d'optimiser et d'étendre certaines parties du processeur.

Sommaire :

- 1) Architecture du projet**
- 2) Implémentations**
- 3) Performances**
- 4) Conclusion**

1) Architecture du projet :

Pour la structure de notre projet nous avons regroupé les fichiers d'en tête (.h) dans un dossier "include", les fichiers sources (.c) dans "src" et les tests unitaires dans "tests". Il y a également un README pour la compréhension du projet et un makefile pour pouvoir exécuter proprement le projet et tester les différentes parties de notre CPU. Nous avons opté pour une approche modulaire plutôt qu'une approche classique car cela nous permettrait de simplifier le débogage et l'optimisation du CPU.

2) Implémentations :

- **hashmap.c** : Ce fichier implémente une table de hachage pour gérer des associations clé-valeur. Il permet d'insérer (*hashmap_insert*), récupérer (*hashmap_get*), supprimer (*hashmap_remove*) et gérer la mémoire (*hashmap_create* / *hashmap_destroy*) des

données. Dans un CPU, cette structure est utile pour gérer les registres (associer nom de registre à valeur) ou optimiser les accès mémoire, en offrant des opérations rapides en temps constant ($O(1)$ en moyenne). La fonction de hachage (*simple_hash*) convertit les clés en indices pour un stockage et une recherche optimisés.

- **memory_handler.c** : Ce fichier implémente un gestionnaire de mémoire dynamique pour un CPU permettant d'allouer (*create_segment*), libérer et fusionner (*remove_segment*) des segments de mémoire. Il utilise une liste chaînée de segments libres et une hashmap pour les segments alloués avec différentes stratégies d'allocation (*First Fit*, *Best Fit*, *Worst Fit*).
- **parser.c** : Ce fichier implémente un analyseur syntaxique pour permettre au CPU de lire du code assembleur. On y retrouve les différentes étapes pour lire un fichier assembleur comme l'extraction des instructions (*parse_data_instruction*) et des données, l'organisation en une structure exploitable (*parser*) pour le CPU. Il gère également les labels et les adresses mémoire (*parse_code_instruction*).
- **cpu.c** : Ce fichier implémente le processus central qui exécute les instructions du programme. On y retrouve les fonctions d'initialisation du CPU (*cpu_init*) avec ses registres et structures de gestion mémoire, les opérations de base comme le stockage et le chargement de données (*store/load*), et la manipulation de la pile (*push_value/pop_value*). Il gère également l'allocation des segments mémoire pour les données, le code et la pile (*allocate_variables/allocate_code_segment*), ainsi que l'allocation dynamique de mémoire (*alloc_es_segment/free_es_segment*).
- **addressing.c** : Ce fichier implémente les modes d'adressage pour un CPU, c'est-à-dire les différentes façons d'accéder aux données (valeurs immédiates, registres, mémoire). Il joue un rôle crucial dans l'exécution des instructions en déterminant où et comment lire/écrire les données.
- **instructions.c** : Ce fichier implémente le jeu d'instructions qui seront exécutées par le CPU lors de la simulation. On y retrouve les fonctions pour gérer chaque type d'instruction comme les transferts de données (*handle_MOV*), les opérations arithmétiques (*ADD*), les comparaisons (*CMP*) et les sauts conditionnels (*JMP*, *JZ*, *JNZ*). La fonction centrale *handle_instruction* fait le dispatch selon le mnémonique de l'instruction reçue. Le fichier contient également les mécanismes pour résoudre les adresses des opérandes (via *execute_instruction*) et pour le cycle fetch-execute avec

fetch_next_instruction qui récupère la prochaine instruction à exécuter depuis le segment de code.

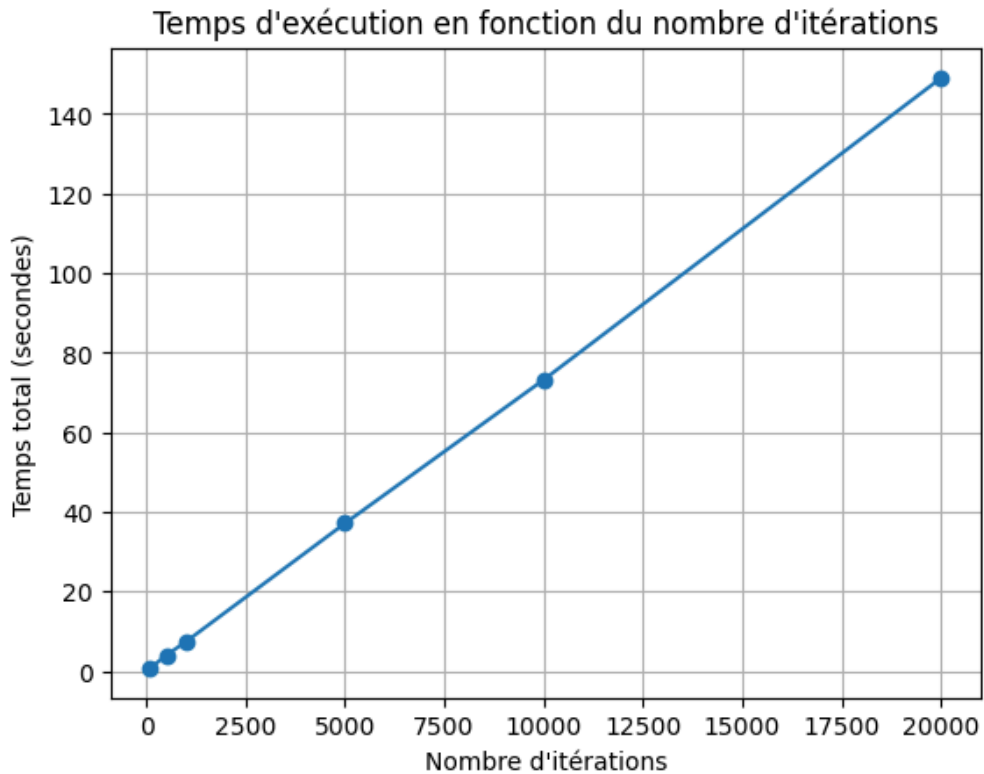
- Dans le fichier **addressing.c** on a fait appel à la bibliothèque **regex.h** et cela nous a amené à implémenter une fonction **matches** afin de vérifier si une chaîne correspond à un motif regex. Cette fonction sera ensuite utilisée dans la fonction **is_number** afin de vérifier si la chaîne est un nombre.
- Dans le fichier **memory_handler.c** nous avons ajouté une fonction qui permet de nettoyer une mémoire **memory_destroy**.
- Dans le fichier **parser.c** nous avons implémenté une fonction **remove_comments** qui permet de supprimer les commentaires d'une ligne. Cette fonction contribue à la sensibilité du parser.

3) Performances :

Nous avons choisi de concentrer notre étude sur la hashmap, le parser et le CPU, car ces modules sont à la fois les plus sollicités en exécution et les plus sensibles aux variations de performances. Bien que d'autres parties comme la gestion de la mémoire, des instructions ou de la pile auraient aussi mérité une analyse approfondie, nous avons préféré cibler les composants les plus représentatifs des performances globales du simulateur.

- **Hashmap** : Une implémentation simple d'où son nom simple_hash. En effet, il y a la possibilité d'optimiser le hashmap en évitant davantage les collisions. Pour cela, on pourrait envisager d'augmenter la taille de la table dynamiquement au lieu d'avoir une taille fixe (ici 128). On pourrait également envisager un autre type de sondage comme le double hachage ou bien utiliser une technique de brassage des clés afin d'améliorer l'uniformité et réduire les collisions.
- **Parser** : On pourrait toujours le rendre plus optimal car il reste sensible à la casse. On pourrait aller plus loin en normalisant la saisie (fonctions annexes, majuscule, minuscule). Nous avons testé le parser sur un échantillon de comparaison de plus en plus grand : ~ 0.000205 sec pour 1000 itérations, ~0.000376 sec pour 2000 itérations et ~ 0.000435 sec pour 10000 itérations. On remarque une progression linéaire en complexité pire cas $O(n)$.
- **Cpu** : Notre processeur démontre des performances linéaires $O(n)$ pour l'exécution des instructions, avec un temps d'exécution négligeable pour des programmes simples $O(1)$.

Les tests avec Valgrind confirment l'absence de fuites mémoire critiques bien que certaines allocations résiduelles puissent encore être optimisées. L'usage d'une hashmap pour les registres et une gestion segmentée de la mémoire garantissent des accès rapides mais des améliorations sont possibles (comme un redimensionnement dynamique de la table de hachage).



Après avoir fait un main où l'on note les mesures de la performance de notre cpu sur plusieurs itérations dans un fichier.csv. Sans trop de surprise, on voit une progression linéaire du temps d'exécution par rapport au nombre d'itérations. Les performances démontrées par notre simulateur de CPU reste raisonnable pour une table de hachage simple.

Tests unitaires : Nos tests unitaires sont organisés par module fonctionnel et visent à valider le comportement des composants de chaque partie du simulateur de CPU. Chaque module dispose d'un test unitaire pour garantir la robustesse et la fiabilité du code :

-
- **test_addressing.c** : Le fichier de test initialise un environnement minimal avec la fonction `setup_test_environment`, une table de registres (AX, BX, etc.) et un segment mémoire DS. Il vérifie le bon fonctionnement des quatre modes d'adressage implémentés : immédiat (42) registre (AX) direct mémoire ([5]) indirect via registre

([AX])). Ce test valide que l'accès aux opérandes via différentes syntaxes est correctement interprété par le CPU.

- **test_cpu.c** : Un test spécifique mesure le temps d'initialisation et de destruction d'un CPU avec un segment mémoire de 1 Mo. Il utilise `clock()` pour chronométrer chaque opération : le temps d'initialisation et le temps de libération mémoire va nous permettre d'avoir une évaluation des performances en termes de scalabilité du cœur du simulateur.
- **test_hashmap.c** : Ce test vérifie les opérations principales : Insertion de paires clé-valeur (`cle1, cle2`), lecture et affichage, suppression et vérification post-suppression, libération mémoire. Il teste les principales fonctions de la structure hashmap et vérifie si le code fonctionne de manière cohérente.
- **test_memory.c** : Ce test vérifie principalement les fonctions du gestionnaire de mémoire. On a implémenté différentes fonctions pour afficher l'état de la mémoire (*`print_known_allocations`, `print_free_segments`, `print_memory_state`*) et une dernière pour tester les différentes stratégies (*`test_allocation_strategies`*).
- **test_parser.c** : Le fichier de test lit un fichier `exemple.asm` et affiche le nombre d'instructions dans `.DATA` et `.CODE`. Les labels reconnus avec leur adresse. Ce test permet de s'assurer que le parser convertit correctement le code assembleur en une structure interprétable par le simulateur.

Notre ***exemple.asm*** contient 4 instructions `.DATA` et 30 instructions `.CODE` parmi les 60 lignes de code assembleur et qui permet de tester différentes opérations de notre simulateur et par la même occasion tester la sensibilité de notre parser avec des espaces, sauts de ligne, tabulation et retour chariot :

1. Les opérations de pile (PUSH / POP)
2. L'adressage mémoire direct et indirect
3. Les opérations arithmétiques (ADD)
4. Les instructions de saut conditionnel (CMP, JZ, JMP)
5. La boucle de type compteur
6. La lecture / écriture mémoire
7. Et l'arrêt propre avec HALT

4) Conclusion :

Globalement notre simulateur de CPU reste basique mais solide. Les performances de nos implémentations notamment le parser sont en $O(n)$ ce qui est raisonnable par rapport aux contraintes imposées et tant qu'on évite les complexités $O(n^2)$. D'après le rapport de Valgrind il n'y a pas de fuites mémoires ni d'erreurs de syntaxe mais on pourrait optimiser la libération des octets alloués : il reste des allocations non libérées (2,778 allocs, 1,363 frees pour les fichiers sources, et 3,604 allocs, 2,101 frees pour les tests unitaires).